TABLE OF CONTENTS

Baridha 11/23

BASIC THEORY:

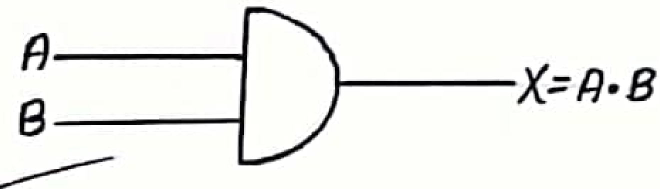
AND Gate (1.1)

AND gate accepts two signals. If the two input values for an AND gate are both 1, the output is 1; otherwise the output is 0.

Boolean Expression

$X = A \cdot B$

Logical Diagram Symbol



Truth Table

Inputs		Output
A	B	$X = A \cdot B$
0	0	0
0	1	0
1	0	0
1	1	1

Experiment Diagram:

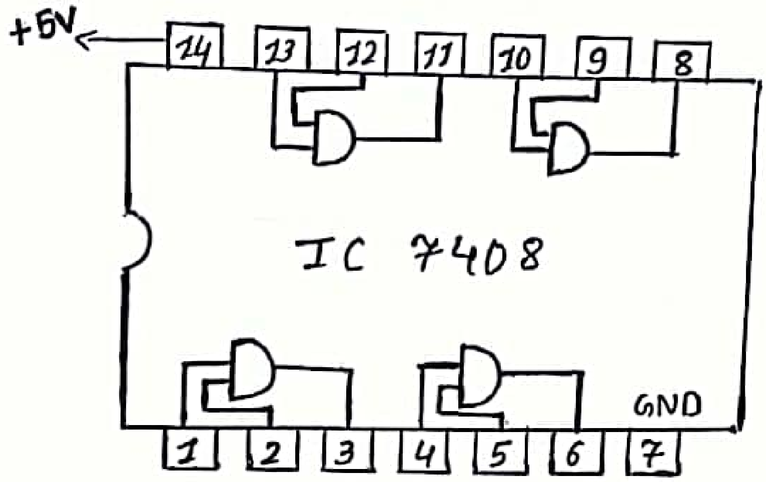


Fig: AND Gate
IC 7408

Observation:
In the experiment with the AND Gate, the output is observed to be high (1) only when both inputs are high (1). For all other combinations of inputs the outputs remains low (0). This confirms the behaviour described by the AND Gates truth table.

Conclusion:
In this way the logical operation of AND Gate was taken place using the lab essentials.

LAB 1 : Experiment with basic Logic Gates

Objective : To understand the working mechanism of basic logic gates such as : AND gate , OR gate, and NOT gate.

Lab Requirements :

a) Bread Board

b) Integrated Circuit (IC)

c) Wire and Wire Cutter

d). LED

e) Power Supply

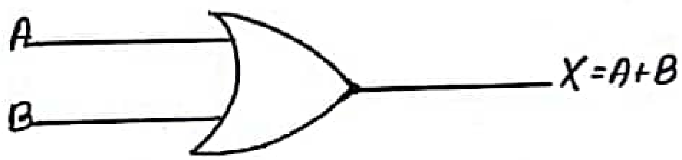
1.2 OR Gate

In an OR gate, if the two input values are both 0, the output value is 0; otherwise the output is 1.

Boolean Expression:

$X = A + B$

Logical Diagram Symbol:



Truth Table

Inputs		Output
A	B	X=A+B
0	0	0
0	1	1
1	0	1
1	1	1

Experiment Diagram:

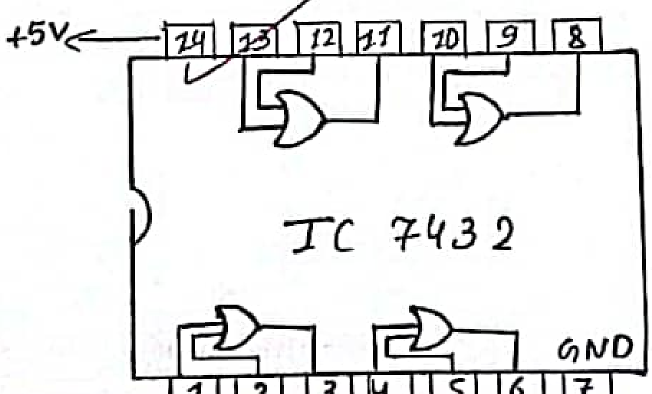


Fig: OR Gate
IC: 7432

Observation :

In the experiment with the OR Gate, the output is observed to be high if at least one of the input is high otherwise it will produce low output when both inputs are low. This reflects the behaviour shown in the OR Gate's truth table.

Conclusion:

In this way, the logic operation of OR Gate was performed in the lab using the equipments required to test it.

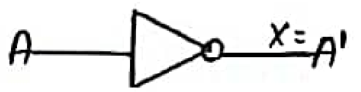
1.3 NOT Gate

A NOT Gate accepts one input value & produces one output value.

Boolean Expression

$X = A'$

Circuit Diagram



Truth Table

Input	Output
A	X=A'
0	1
1	0

Experiment Diagram:

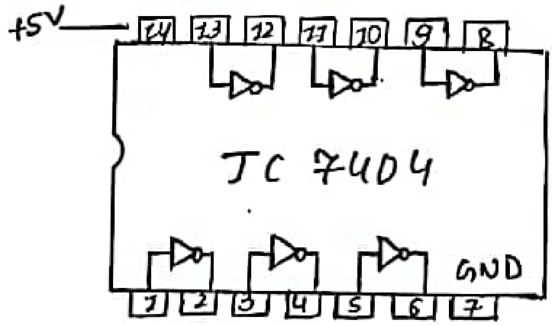


Fig: NOT Gate
IC : 7404

Observation:

In the experiment with the NOT Gate, the output is observed to be the inverse of the input. When the input is high, the output is low and vice-versa. This behaviour reflects the NOT Gate's truth table.

Conclusion:

In this way, the logical operation of NOT gate was carried out.

LAB 2 : Experiment with Universal Gates

Objective: To understand the working mechanism of universal gates i.e. NAND & NOR Gates.

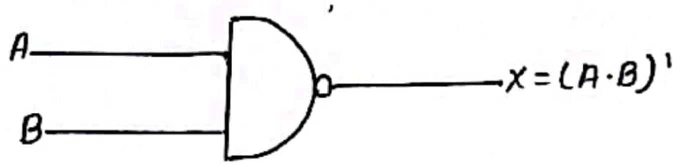
2.1 NAND Gate

NAND gate is the combination of NOT gate and AND gate. If the two input values for an NAND gate are both 1, the output is 0; otherwise the output is 1.

Boolean Expression

$$X = (A \cdot B)'$$

Logic Diagram Symbol



Truth Table

Inputs		Output
A	B	$X=(A\cdot B)'$
0	0	1
0	1	1
1	0	1
1	1	0

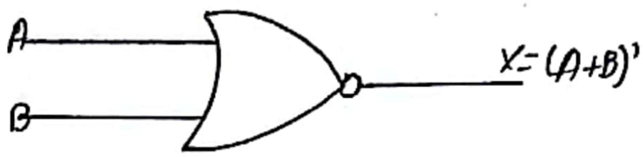
2.2 NOR Gate

NOR Gate is the combination of NOT gate and OR Gate. If the two Input values for NOR gate are both 0, output value is 1; otherwise the output is 0.

Boolean Expression

$X = (A+B)'$

Logical Diagram Symbol



Truth Table

Inputs		Output
A	B	$X=(A+B)'$
0	0	1
0	1	0
1	0	0
1	1	0

Experiment Diagram

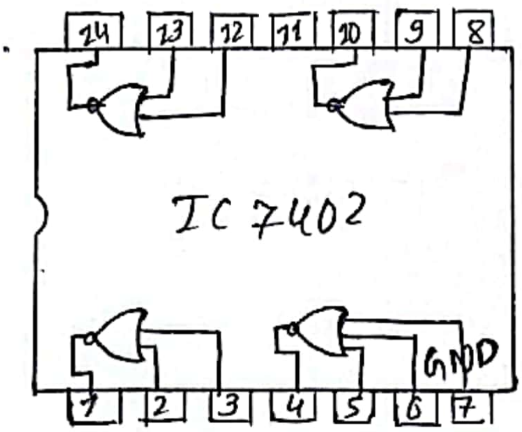


Fig : NOR Gate IC: 7402

Experiment Diagram:

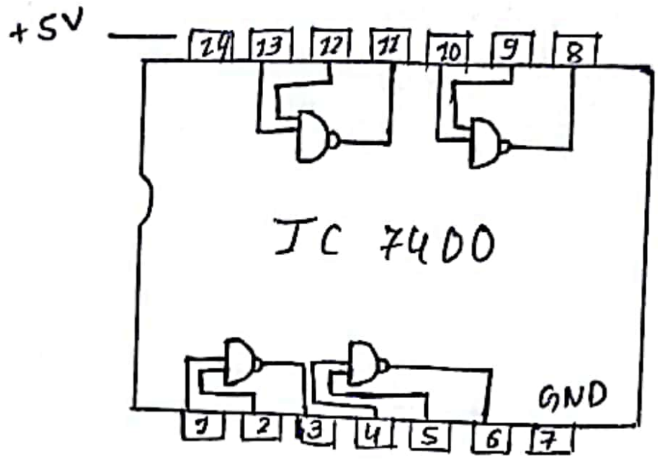


Fig: NAND Gate
IC: 7400

Observation:

In the experiment with NAND Gate, the output is observed to be low only when both inputs are high. For all other combinations of inputs, the output is high. This behaviour is shown by the NAND Gate's truth table as well, which is the inverse of AND Gate.

Conclusion:
In this way, the logical operation of NAND gate was initiated.

Observation :

In this experiment with the NOR gate, the output is observed to be high only when both inputs are low. For all other input combinations, the output is low. This reflects the NOR Gate's truth table, which is the inverse of OR Gate.

Conclusion :

So, in this way, the logic operation of NOR gate was performed.

LAB 3 : Experiment with Derived Gates

Objective: To understand the working mechanism of logic gates i.e. X-OR and X-NOR Gates.

3.1 X-OR Gate
(Exclusive-OR Gate)

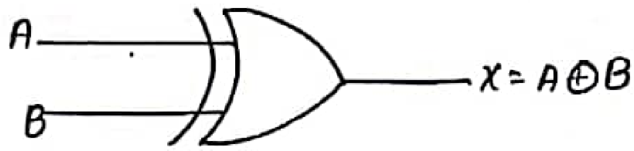
An X-OR Gate produces 0 if its two inputs are the same, and a 1 otherwise.

Boolean Expression

$X = A \oplus B$

$= AB' + A'B$

Logical Graphical Symbol



Truth Table

A	B	A'	B'	AB'	A'B	X = AB'+A'B
0	0	1	1	0	0	0
0	1	1	0	0	1	1
1	0	0	1	1	0	1
1	1	0	0	0	0	0

Experiment Diagram:

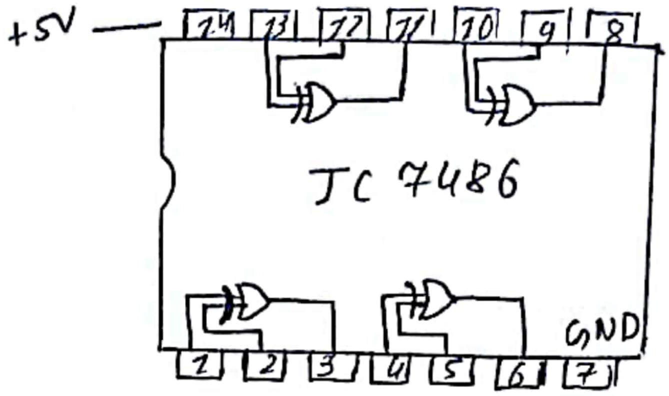


Fig: ~~N~~XOR Gate
IC: 7486

Observation:

In the experiment with X-OR Gate, the output is observed to be high only when the inputs are different. When both inputs are same, the output is low. This matches the X-OR's truth table.

Conclusion:

In this way, the experiment of X-OR Gate was verified.

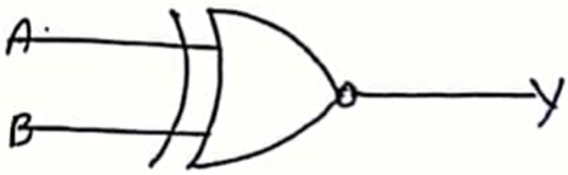
7.2 X-NOR Gate
(Exclusive NOR Gate)

X-NOR Gate is the component of X-OR. An X-NOR Gate produces 1 if its two inputs are the same and a 0 otherwise.

Boolean Expression

$$X = \overline{A \oplus B}$$
$$= A \odot B$$
$$Y = (\overline{A \oplus B}) = (A.B + A'.B')$$

Logical Diagram Symbol



Truth Table

A	B	A'	B'	A.B	A'.B'	Y = AB+A'B'
0	0	1	1	0	1	1
0	1	1	0	0	0	0
1	0	0	1	0	0	0
1	1	0	0	1	0	1

Experiment Diagram

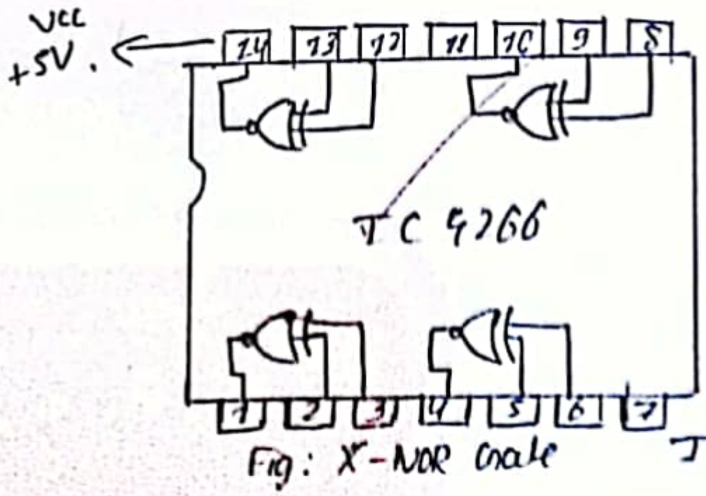


Fig: X-NOR Gate IC: 4266

Observation:

In the above experiment with X-NOR Gate the output is observed to be high when both inputs are the same, when both inputs are different then the output is low. This confirm that the X-NOR Gate's truth table, which is the inverse of X-OR Gate.

Conclusion:

In this way, the logic operation of X-NOR was initiated during the experiment.

LAB 4 : Experiment of Adder Circuit

Objectives: To verify the operation of Half-Adder and Full Adder.

Lab Requirements: Bread Board, Integrated Circuit (IC), Power Supply, Wire and wire cutter and LED.

Components: 7486 (X-OR), 7404 (Inverter), 7408 (AND), 7432 (OR), LED

4.1 Half Adder

Half Adder is a combinational logic circuit. It is used for adding two single bit binary numbers. It has two outputs; one is sum and another is carry.

Boolean Expression

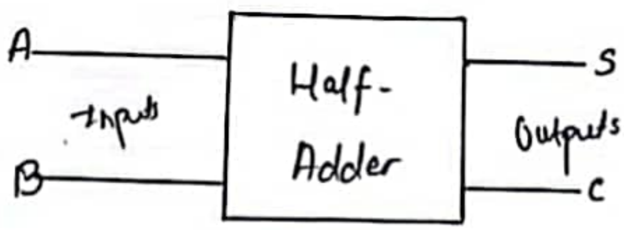
$Sum(s) = A \oplus B$

$= A'B + AB'$

$Carry(c) = A \cdot B$

Block Diagram

~~Carry (c) = A·B~~



Truth Table of Half Adder

Inputs		Outputs	
A	B	Sum (s)	Carry (c)
0	0	0	0
0	1	1	0
1	0	1	0
1	1	0	1

Logical Circuit:

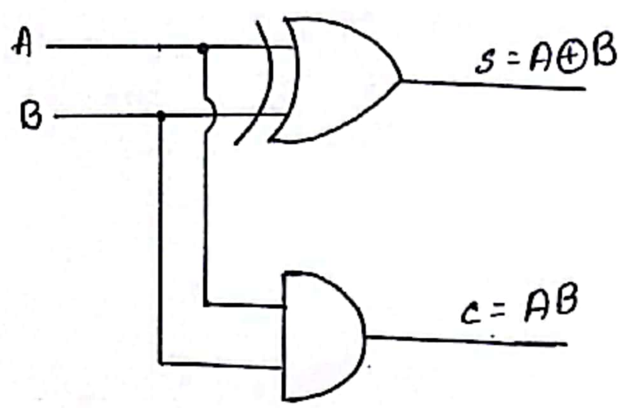


Fig: logical Diagram of Half Adder

4.2 Full Adder

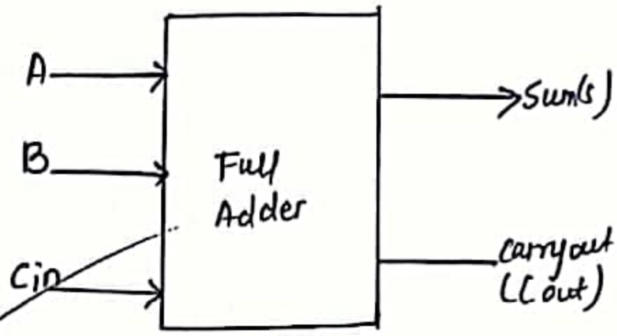
Full Adder is the adder that adds three inputs and produces two outputs. The first two inputs are A and B and the third input is an input carry as C-In. The output carry is designated as C-Out and the normal output is designated as S which is SUM.

Boolean Expression

Sum(S) = $A \oplus B \oplus C_{in}$

Carry ~~(c)~~ out (C out) = $AB + (A \oplus B)C_{in}$

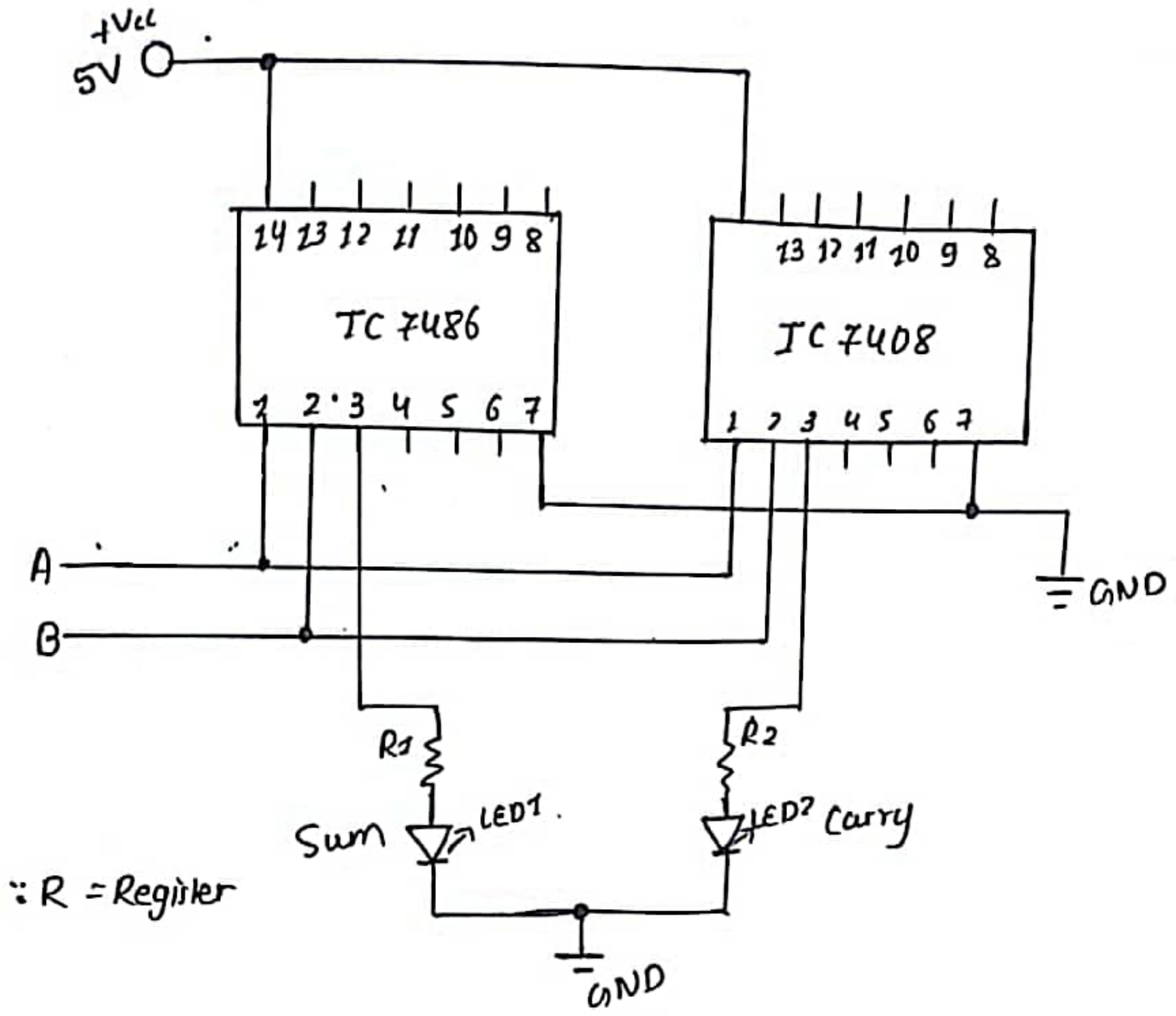
Block Diagram



Truth Table

Inputs			Outputs	
A	B	C in	Sum (s)	Carry (c)
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1

Experiment Diagram

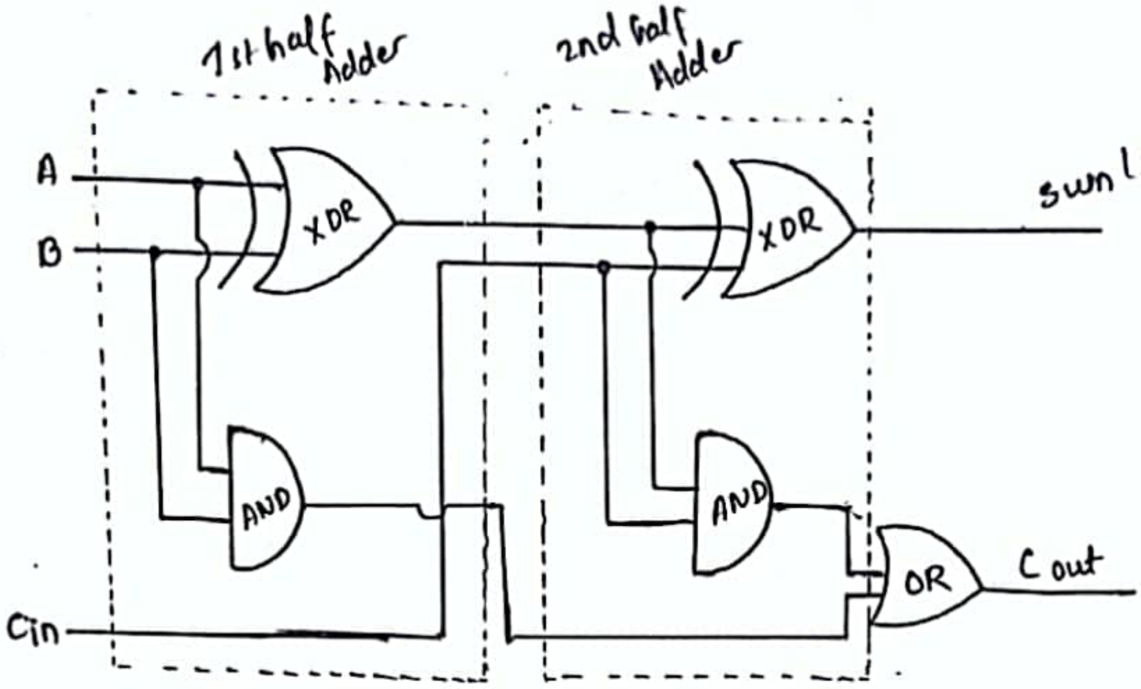


Observation :

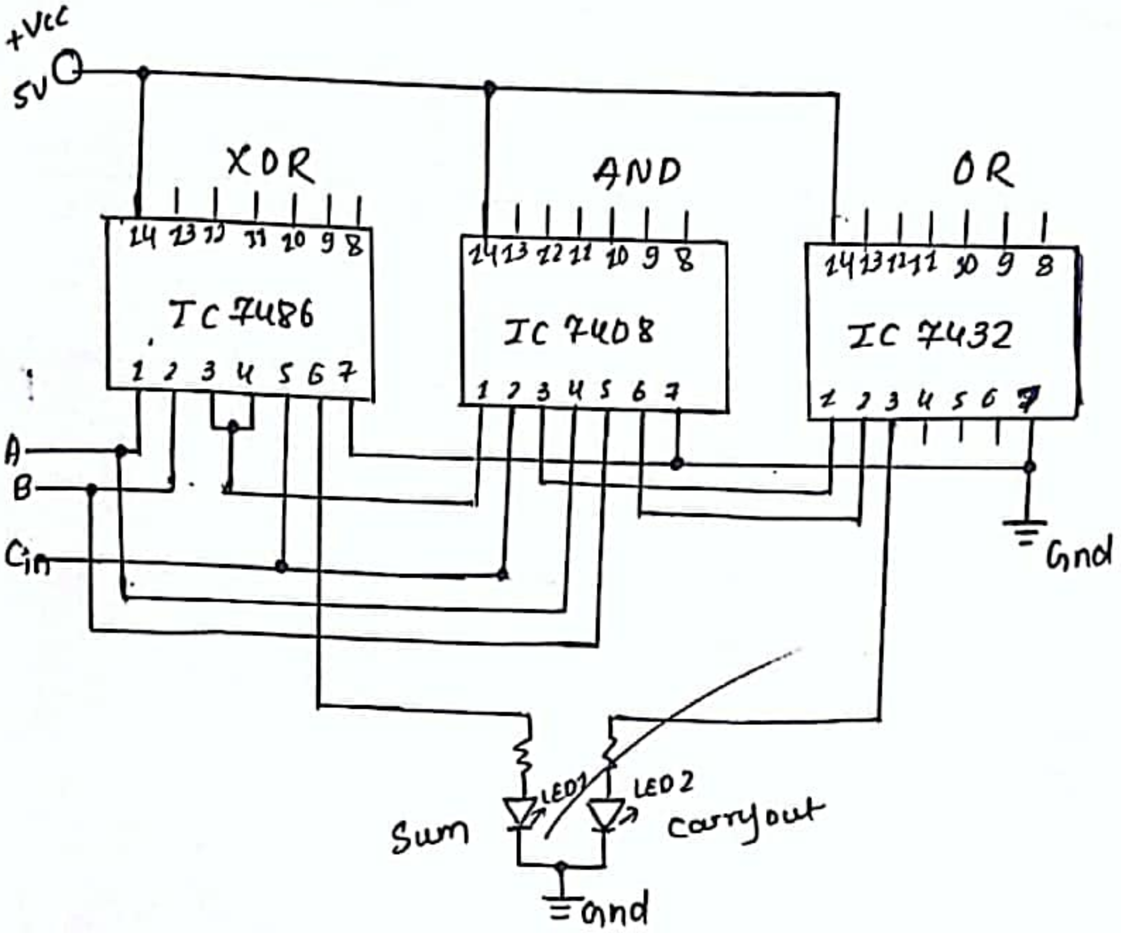
In this experiment with half adder, the circuit successfully perform binary operation (addition) on the two single-bit inputs. The sum (s) output, connected to an LED lights ~~glows~~ up when the inputs are different, following the behaviour of an X-OR gate. The carry (c) output, connected to another LED, lights up when both inputs are high, as expected from the AND Gate operation. This confirms the correct functioning of the half-adder, with the LEDs indicating respective outputs.

Conclusion: Here, Half Adder worked when the sum (s) LED lit up for different inputs & carry (c) LED lit up when both inputs were high.

Implementation of Full Adder with two half adders and an OR Gate



Experiment Diagram



Observation :

The above experiment depicts a Full Adder circuit using ICs 7486 (XOR) gates, 7408 (AND) gates, and 7432 (OR) gates. The inputs A, B and Cin are processed through these gates to produce the Sum and Carry-out outputs. The XOR gates compute the sum, while the AND and OR gates generates the carry out.

Conclusion :

LAB 5: Experiment of Subtractor Circuit

Objectives: To verify the operation of Half-Subtractor and ~~Hal~~ Full Subtractor

Lab Requirements: Bread Board, Integrated Circuit (IC), Power Supply, Wire and Wire Cutter and LED

5.1 Half Subtractor

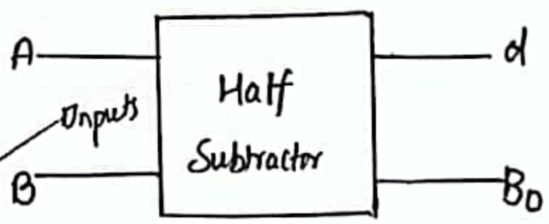
Half Subtractor is a combinational logic circuit which is used to perform subtraction of two input bits and hence provide outputs. It has two inputs and two outputs, one is Borrow (Bo) and another is difference (d).

Boolean Expression

~~Sum~~ Difference $(d) = A \oplus B$
$= A'B + AB'$

Carry $(c) = A' \cdot B$

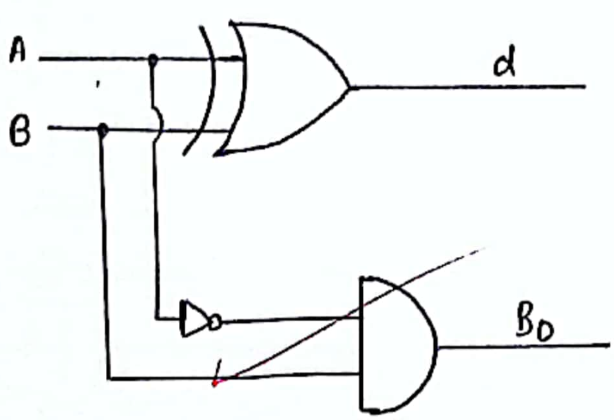
Block Diagram

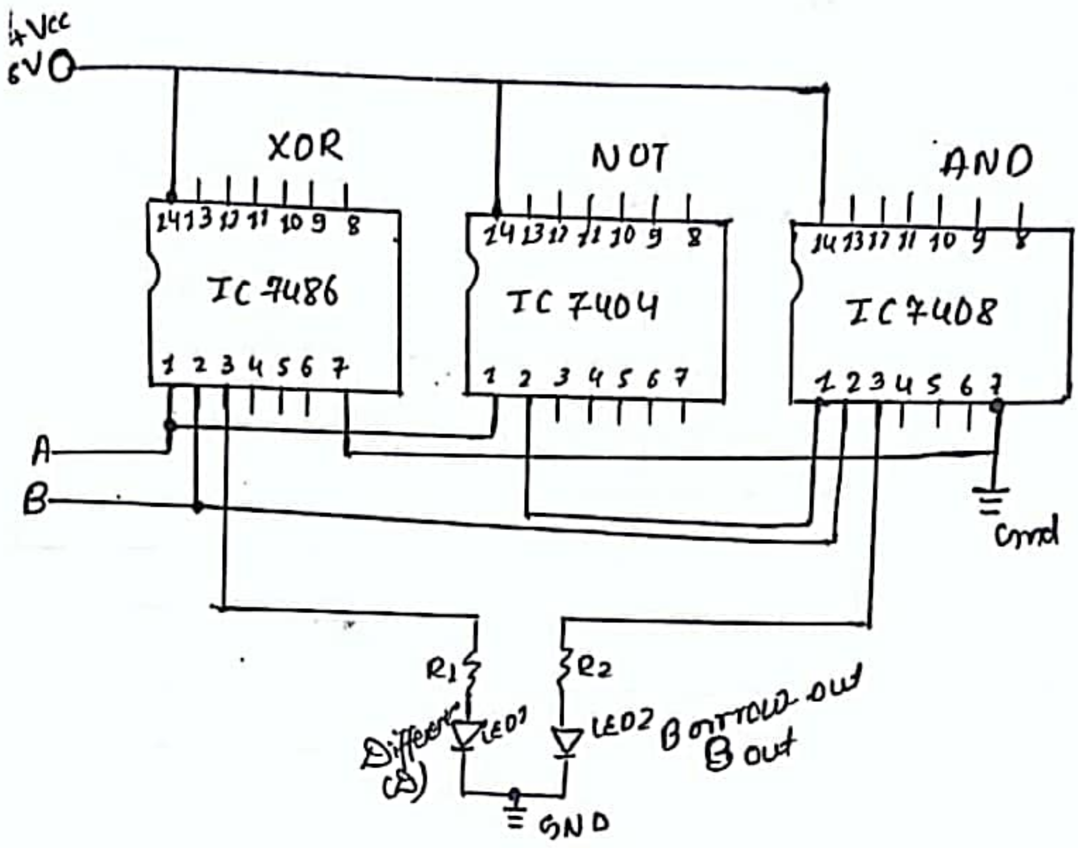


Truth Table of Half Subtractor

Inputs		Outputs	
A	B	Difference(d)	Borrow(Bo)
0	0	0	0
0	1	1	1
1	0	1	0
1	1	0	0

Logical Circuit:





Observation:

In the half subtractor experiment, the Difference (D) LED lit up when the inputs were different, following the XOR gate behaviour. The Borrow (Bo) LED lit up when the second input was high and the first input was low, as expected from the AND gate. This confirms the correct functioning of the half subtractor for basic binary subtraction.

Conclusion:

In this way, we confirm the correct functionality of half-subtractor, as the Difference LED lights up for different inputs and Borrow LED lights up when the first input is lower than second.

5.2 Full Subtractor

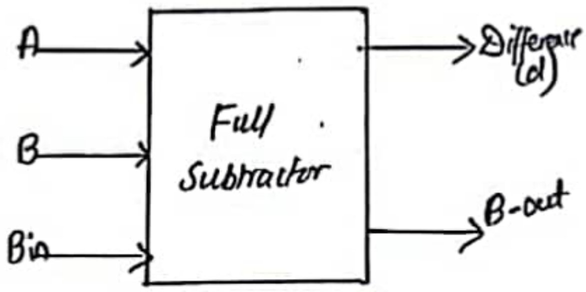
Full Subtractor is the subtractor that subtracts three inputs and produces two outputs. The first two inputs are A and B and the third input is an input borrow Bin. The output borrow is designed as B-Out and the normal output is designated as D which is Difference.

Boolean Expression

$$\text{Difference } (D) = A \oplus B \oplus B_{in}$$

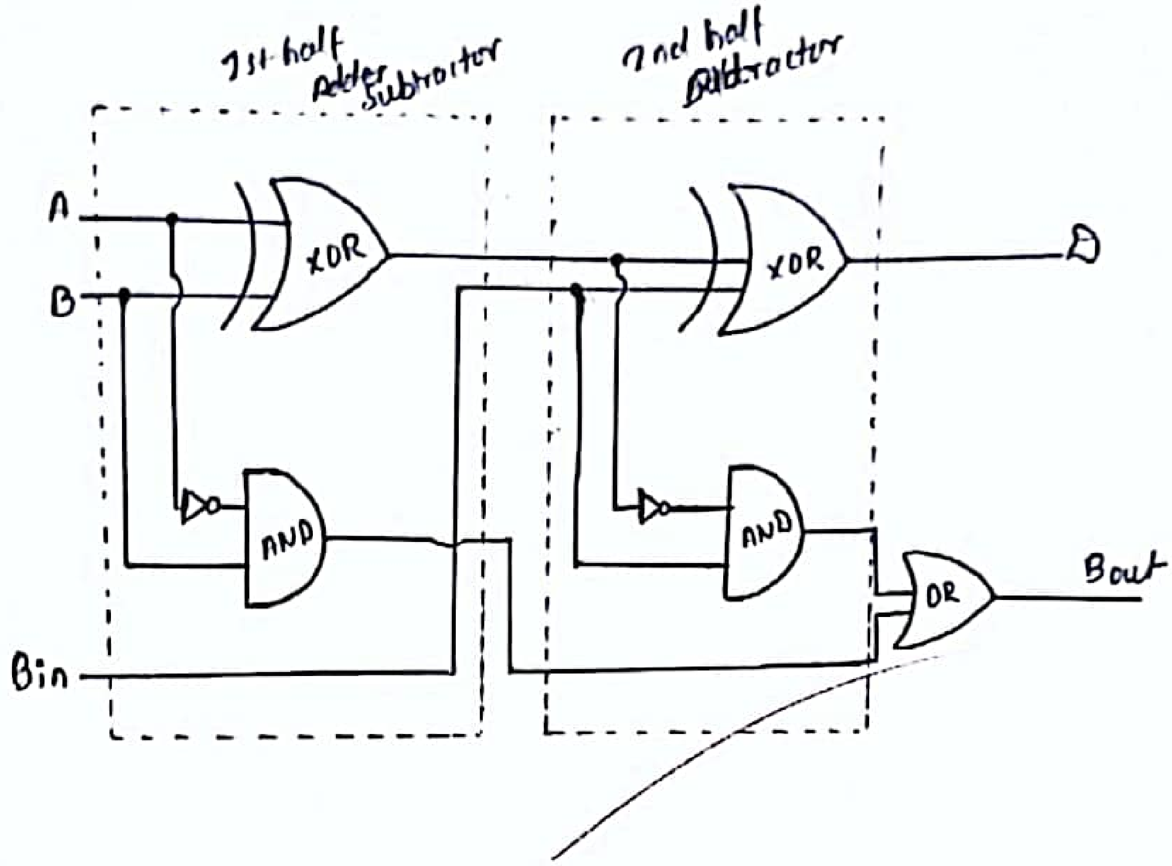
$$\text{Borrow Out } (B_{out}) = A'B\,(A \oplus B)'B_{in}$$

Block Diagram



Inputs			Outputs	
A	B	Bin	(d) Difference	Bout
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1

Implementation of Full Subtractor with two half subtractors and an OR Gate



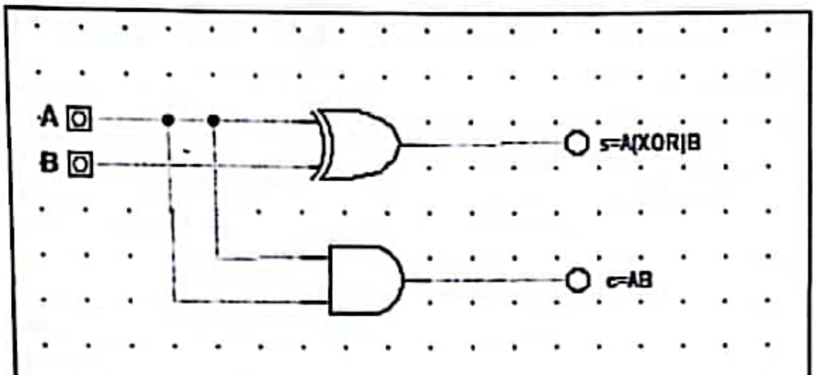


Fig: Half Adder

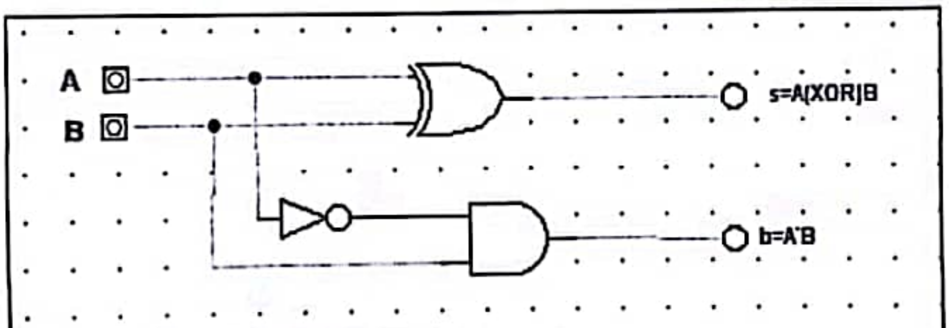


Fig: Half Subtractor

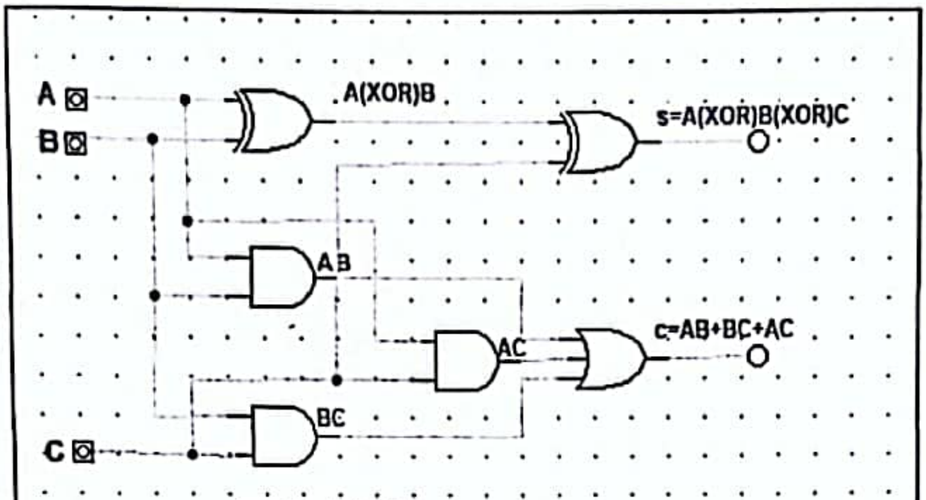


Fig: Full Adder

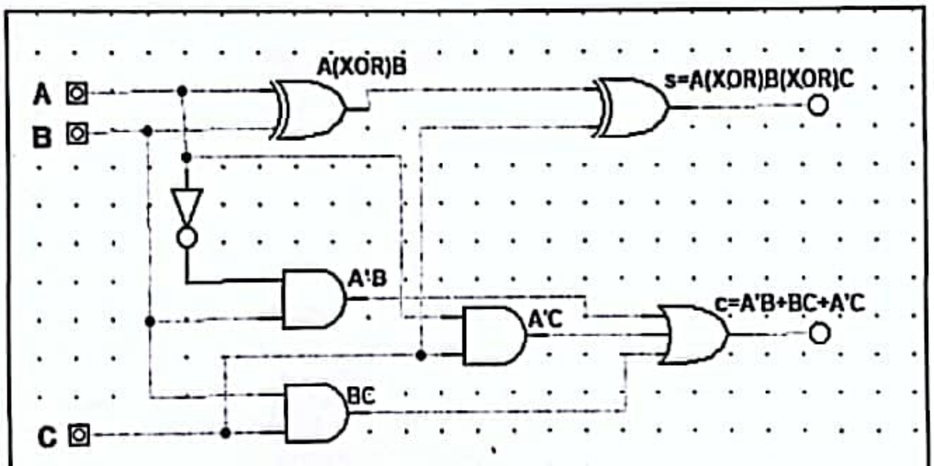


Fig: Full Subtractor

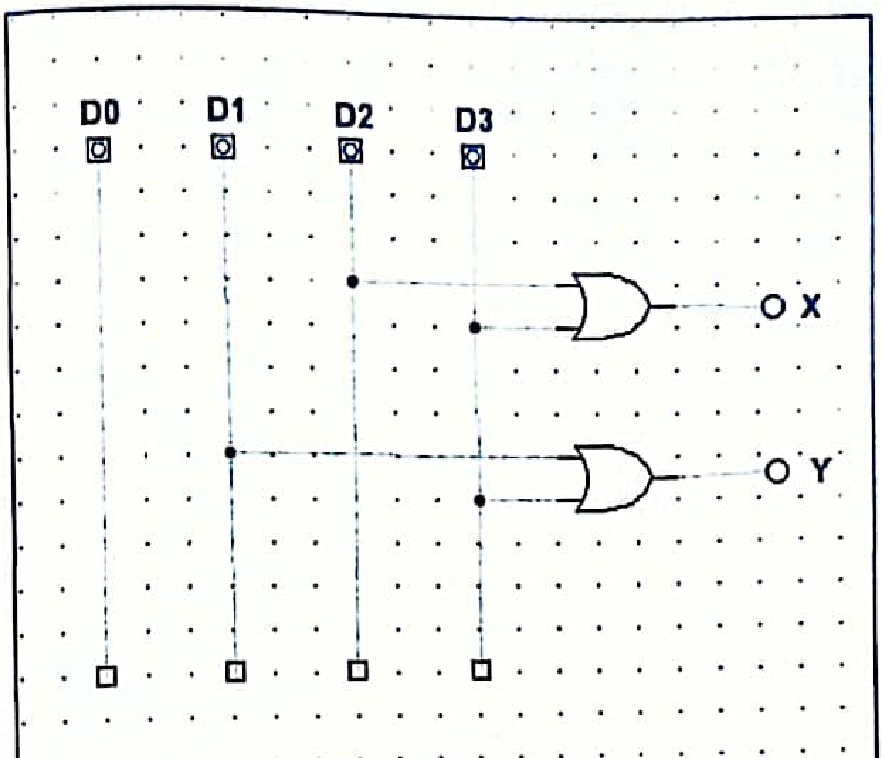


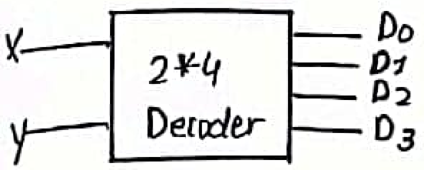
Fig: 4*2 Encoder

6.2 Decoder

It is a combinational circuit and it performs reverse operation of an Encoder. It has n input line and maximum of 2^n output lines.

Example : 2×4 Decoder

Block Diagram



$D_0 = X' \cdot Y'$, $D_1 = X' \cdot Y$
$D_2 = X \cdot Y'$, $D_3 = X \cdot Y$

Truth Table

Inputs		Outputs			
X	Y	D_0	D_1	D_2	D_3
0	0	1	0	0	0
0	1	0	1	0	0
1	0	0	0	1	0
1	1	0	0	0	1

Logic Circuit Diagram:

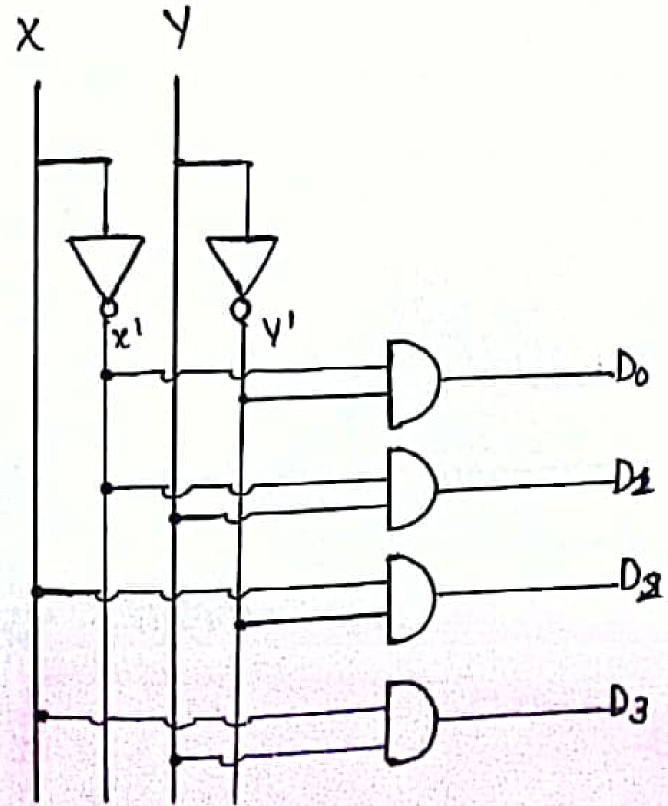


Fig : logic circuit of 2*4 Decoder

LAB 6 : Design Encoder and Decoder

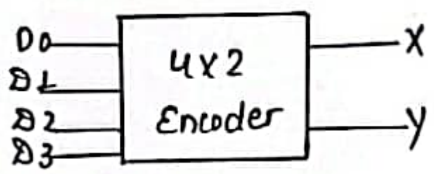
Objective: To understand the concept of Encoder and Decoder

Lab Requirements: Digital Works Application

6.1 Encoder

An encoder is a combinational circuit that performs the inverse operation from that of decoder. It has 2^n input lines and n output lines. The output lines generate the binary code corresponding to the input line.
Example: Design 4X2 Encoder

Block Diagram



Truth Table

Inputs				Outputs	
D_0	D_1	D_2	D_3	X	Y
1	0	0	0	0	0
0	1	0	0	0	1
0	0	1	0	1	0
0	0	0	1	1	1

$X = D_2 + D_3$
$Y = D_1 + D_3$

Logic Circuit Diagram

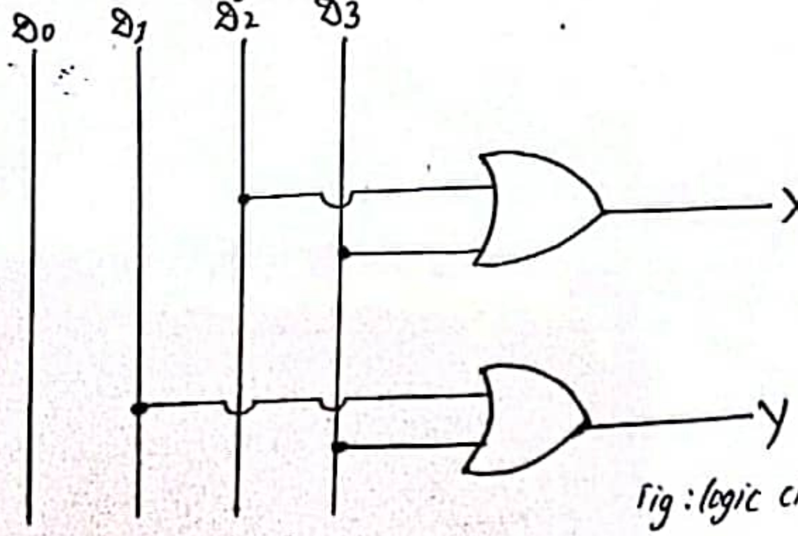


Fig: logic circuit of 4*2 Decoder

LAB 7: Design Multiplexer (MUX) and De-Multiplexer (De-MUX)

Objective: To understand the concept of MUX and De-MUX

Lab Requirement: Digital Work Application

7.1 Multiplexer (MUX)

A multiplexer is a combinational circuit that has many data inputs and a single output, depending on control or select inputs. It has 2^n input line and 'n' selection line and a single output line. Inputs are selected on the basis of selection lines.

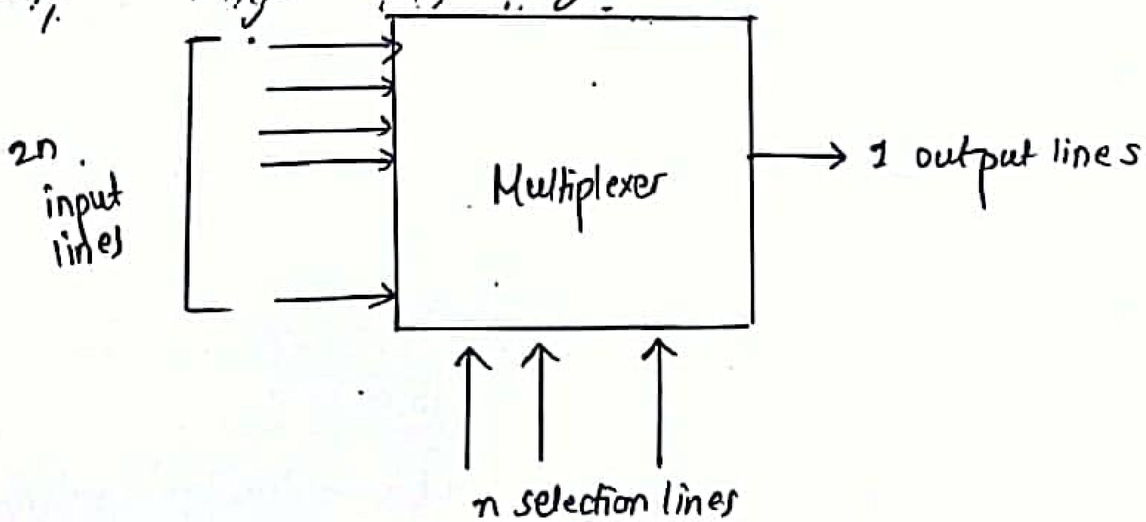


Fig: Block Diagram

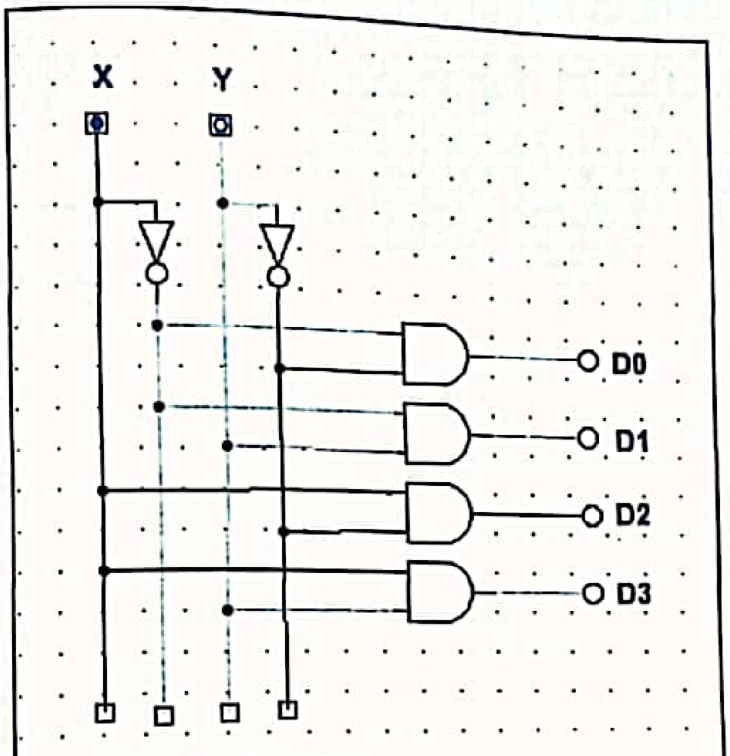


Fig: 2*4 Decoder

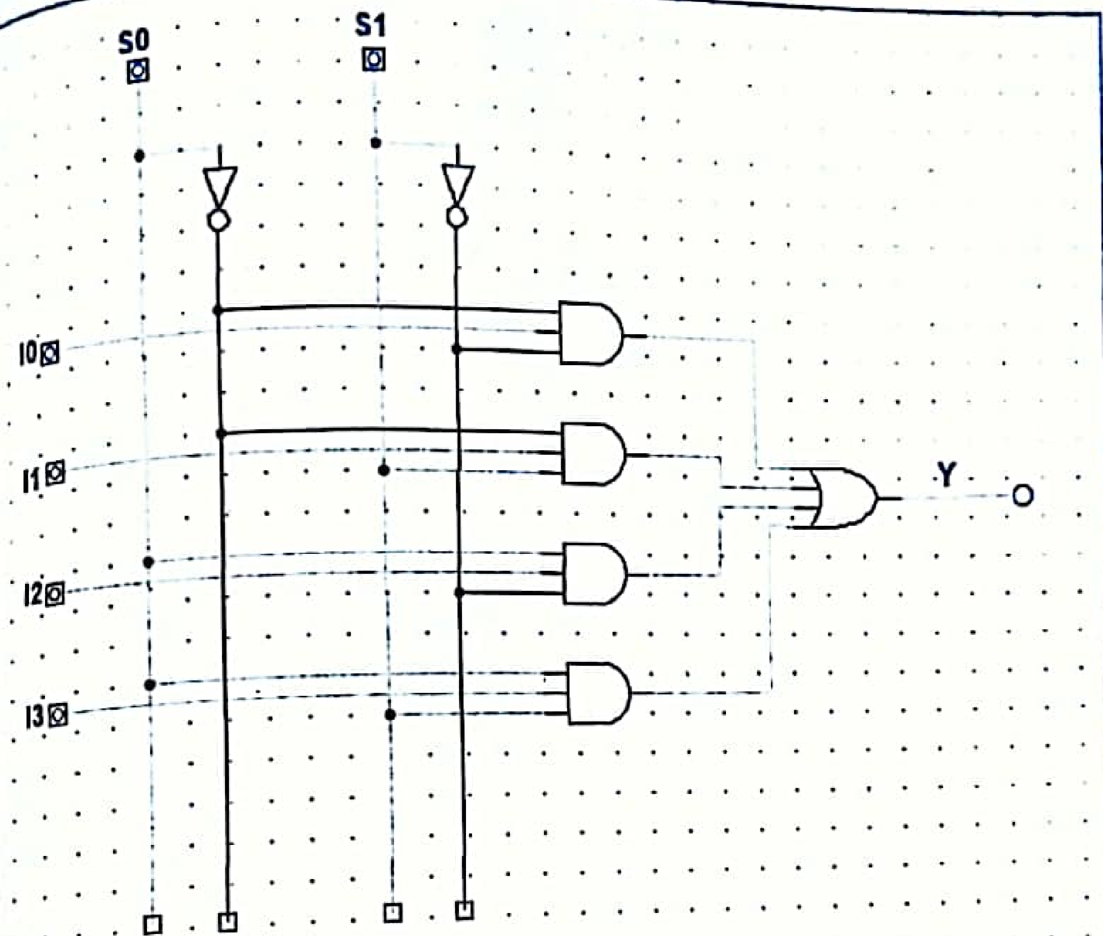
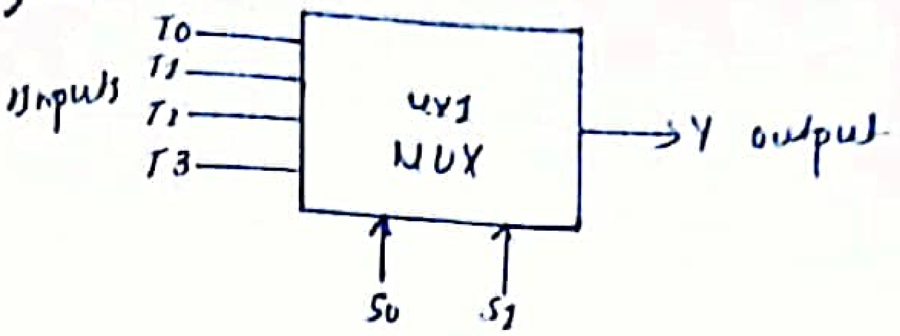


Fig : 4*1 Multiplexer

Example: Design 4x1 MUX

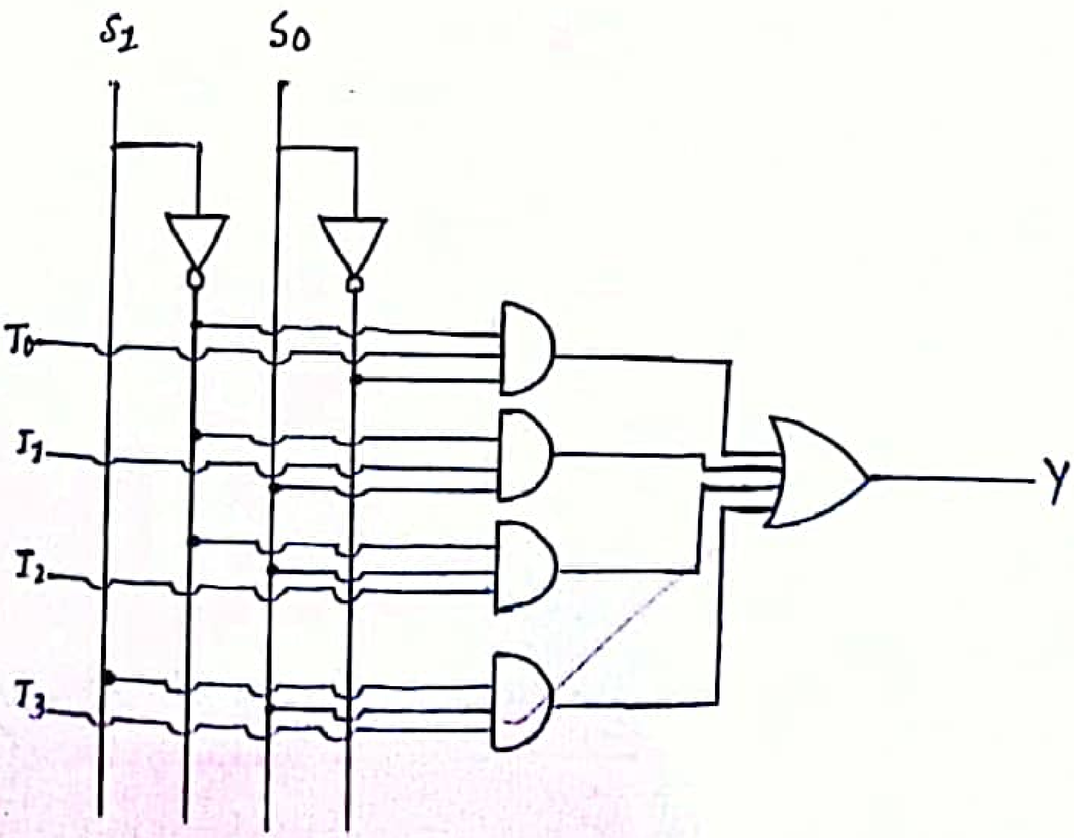
Block Diagram



Function Table

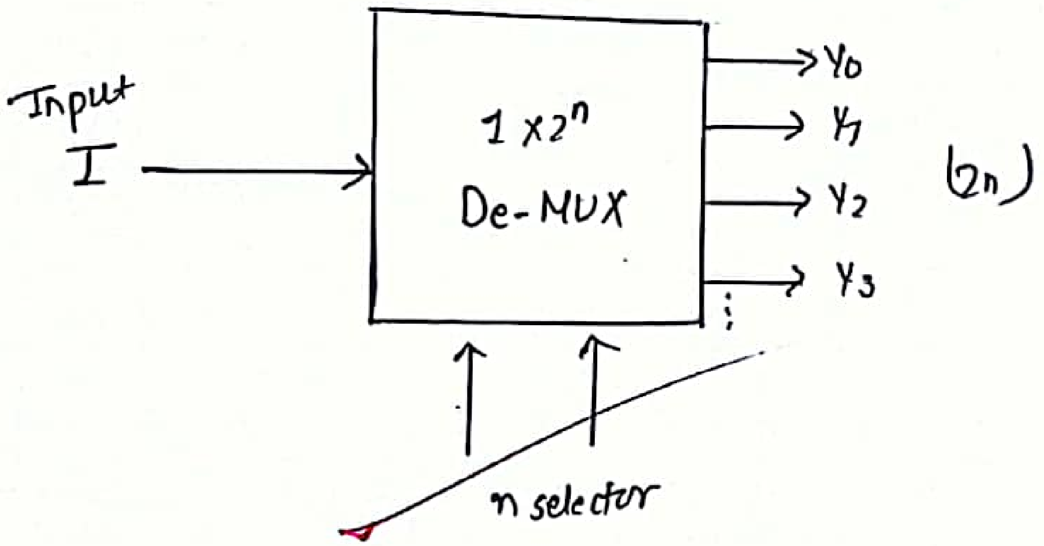
Selection		Output
S_1	S_0	Y
0	0	I_0
0	1	I_1
1	0	I_2
1	1	I_3

Logic Circuit Diagram



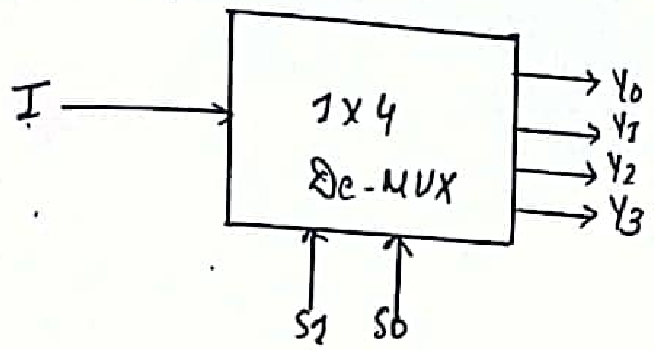
7.2 De-Multiplexer (De-MUX)

De-MUX is a combinational circuit and it performs reverse operation of multiplexer. It has single input line, 'n' selection line and maximum of 2^n output line.



Example: Design 1 x 4 De-MUX

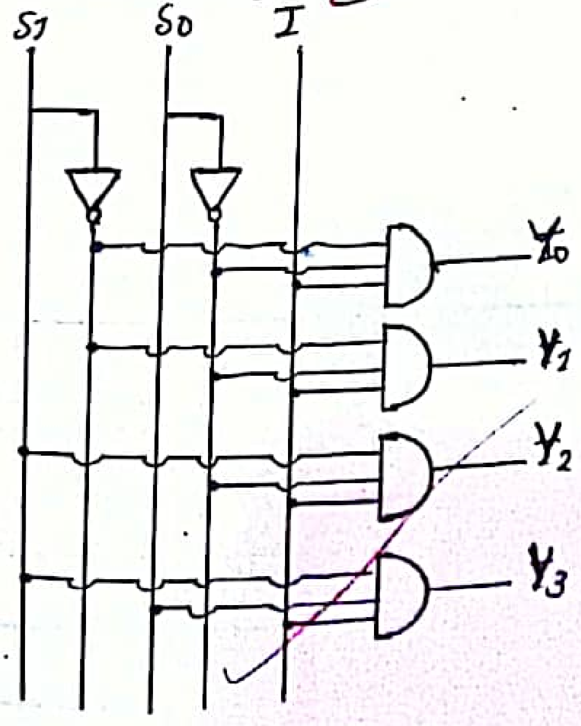
Block Diagram



Function Table

Selection		Outputs			
S_1	S_0	Y_0	Y_1	Y_2	Y_3
0	0	I	0	0	0
0	1	0	I	0	0
1	0	0	0	I	0
1	1	0	0	0	I

logic Circuit Diagram



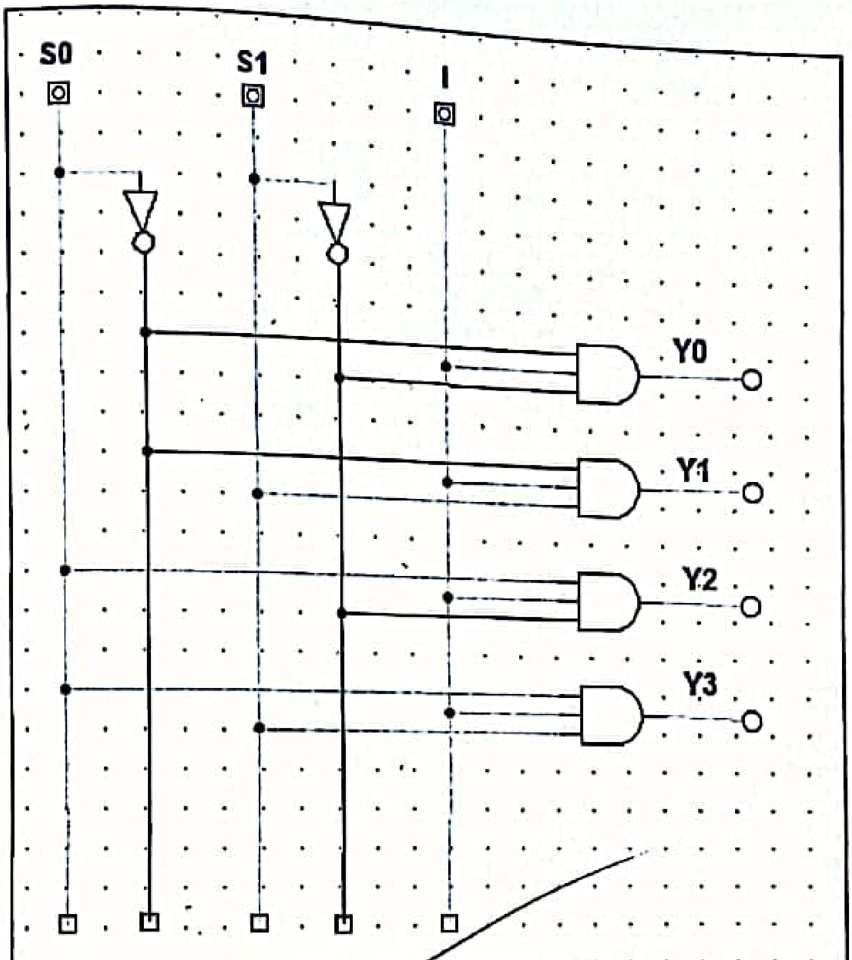


Fig : 1*4 De-Multiplexer

LAB 8: Asynchronous & Synchronous

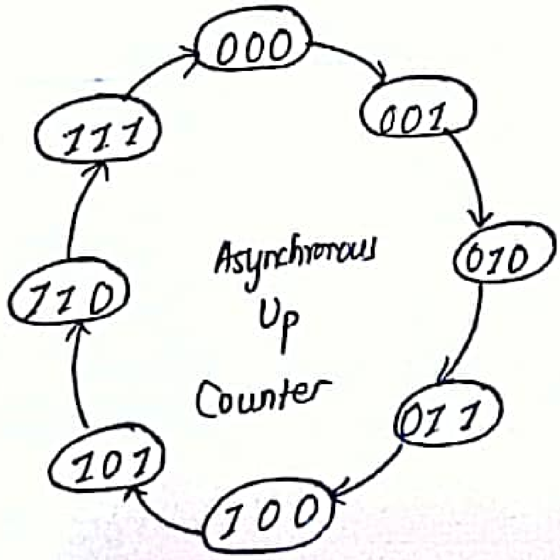
Objective: To study & understand the concept of Asynchronous Counter & Synchronous Counter

Lab Requirements: Digital Works Application

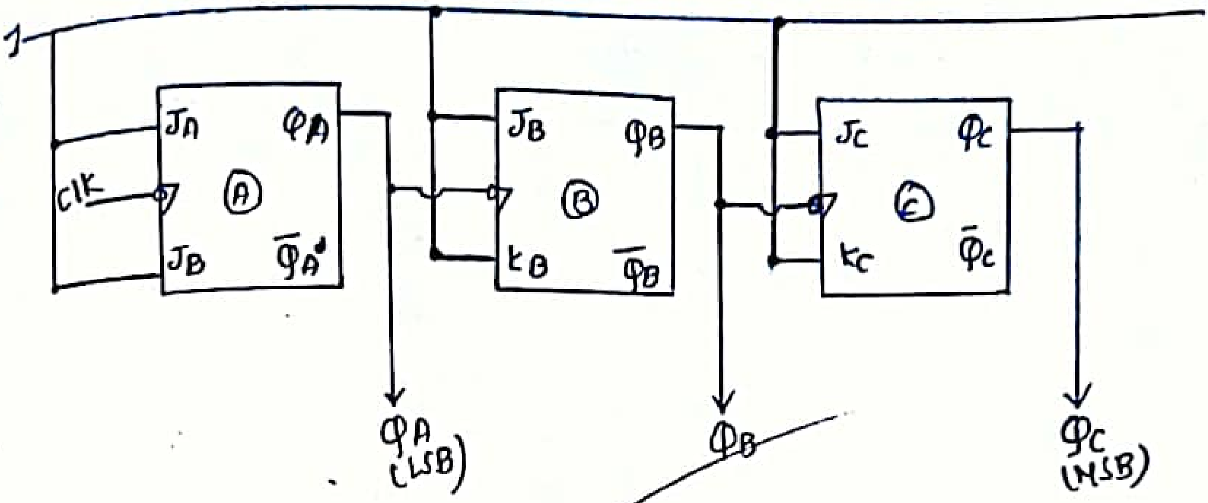
8.1 Asynchronous Counter

In Asynchronous Counter, first flipflop is clocked by external clock after that output of previous flipflop clocks to next flipflop. The propagation delay of the counter is equals to cumilative delay of all the flipflops. It has slow counting and simple design process.

Example: Design 3 bit (MOD 8) Asynchronous Up Counter using JK FlipFlop



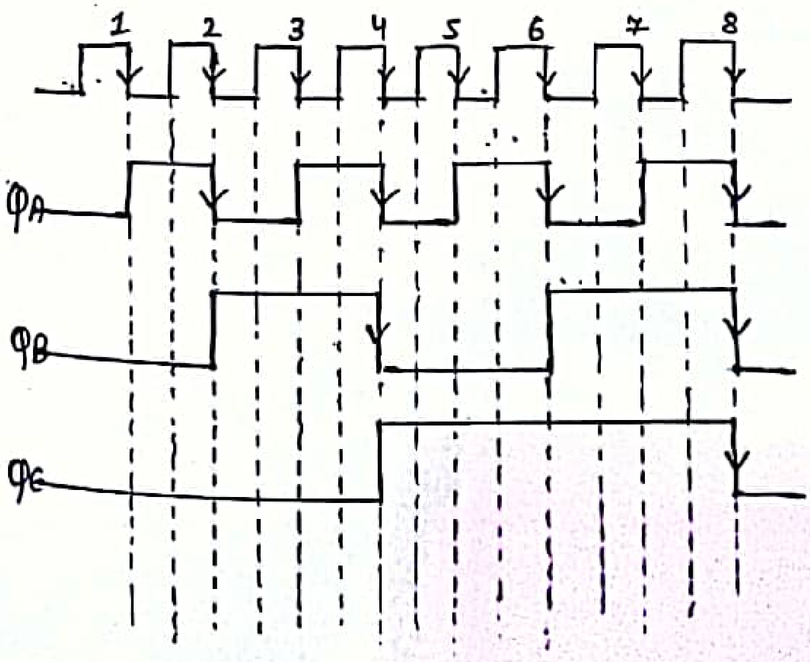
Logic Circuit Diagram :

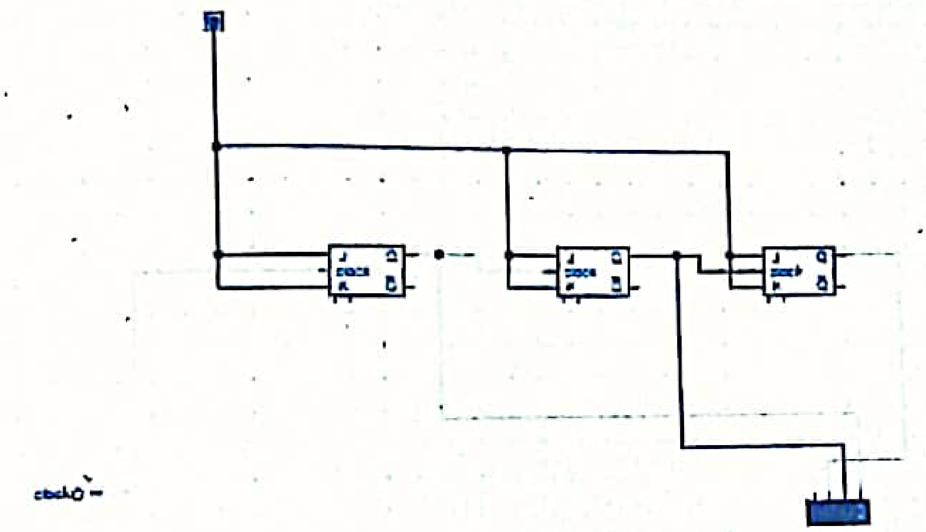


Truth Table

CLK	QC	QB	QA
Initially	0	0	0
1st	0	0	1
2nd	0	1	0
3rd	0	1	1
4th	1	0	0
5th	1	0	1
6th	1	1	0
7th	1	1	1

Timing Diagram :





3 Bit Asynchronous Up Counter

8.2 Synchronous Counter

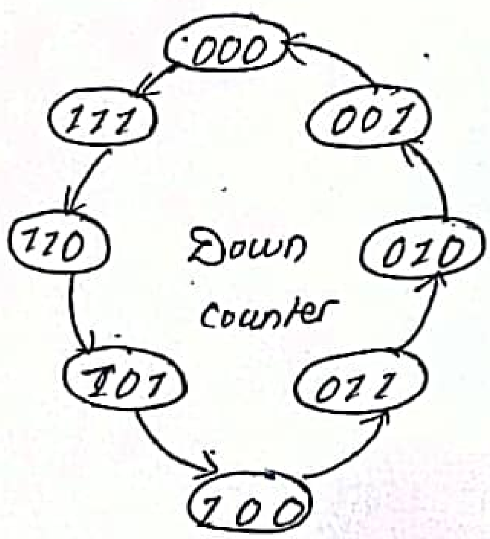
In synchronous counter, all the flipflops are triggered by a common clock pulse simultaneously. Propagation delay of the counter is equals to delay of single flipflop. It is faster and has complex design process & complex circuit diagram.

Example: Design 3 bit (MOD 8) Synchronous Down Counter using T-FlipFlop

Excitation Table of T-FlipFlop

Q_n	Q_{n+1}	T
0	0	0
0	1	1
1	0	1
1	1	0

State Diagram & Circuit Excitation Table



Present State			Next State			Output		
Q_C	Q_B	Q_A	Q_{C+1}	Q_{B+1}	Q_{A+1}	T_C	T_B	T_A
0	0	0	1	1	1	1	1	1
0	0	1	0	0	0	0	0	1
0	1	0	0	0	1	0	1	1
0	1	1	0	1	0	0	0	1
1	0	0	0	1	1	1	1	1
1	0	1	1	0	0	0	0	1
1	1	0	1	0	1	0	1	1
1	1	1	1	1	0	0	0	1

K-Map for the Output

For T_C

$Q_C \backslash Q_BQ_A$	00	01	11	10
0	1			
1	1			

$G_1 \Rightarrow \left.\begin{matrix}000\\100\end{matrix}\right\} \bar{Q}_B\bar{Q}_A$

$T_C = \bar{Q}_B\bar{Q}_A$

For T_B

$Q_C \backslash Q_BQ_A$	00	01	11	10
0	1			1
1	1			1

$G_1 \rightarrow \left.\begin{matrix}000\\010\\100\\110\end{matrix}\right\} \bar{Q}_A$

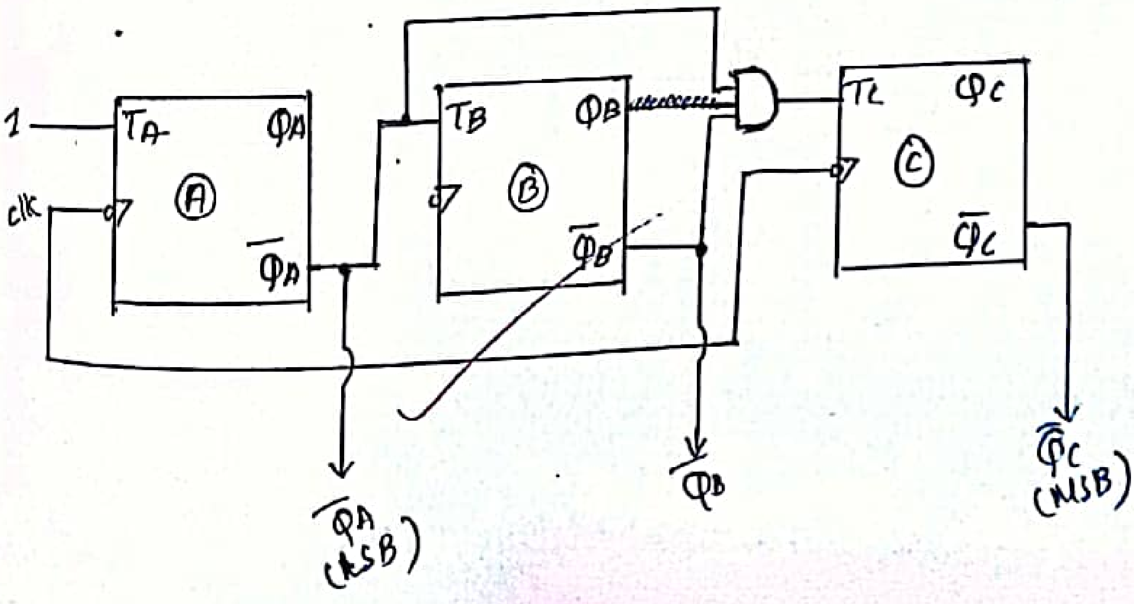
$T_B = \bar{Q}_A$

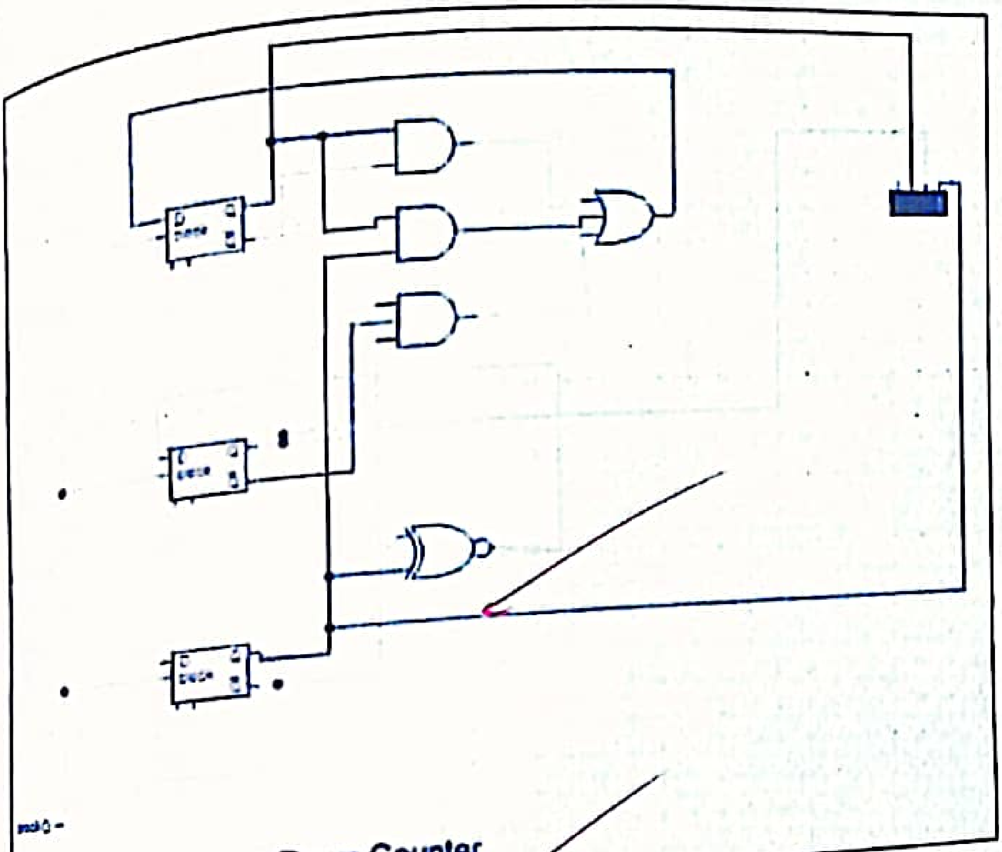
For T_A

$Q_C \backslash Q_BQ_A$	00	01	11	10
0	1	1	1	1
1	1	1	1	1

$T_A = 1$

Logic Circuit Diagram





3 Bit Synchronous Down Counter

LAB 9 : Shift Registers

9.1 Serial In Serial Out (SISO):

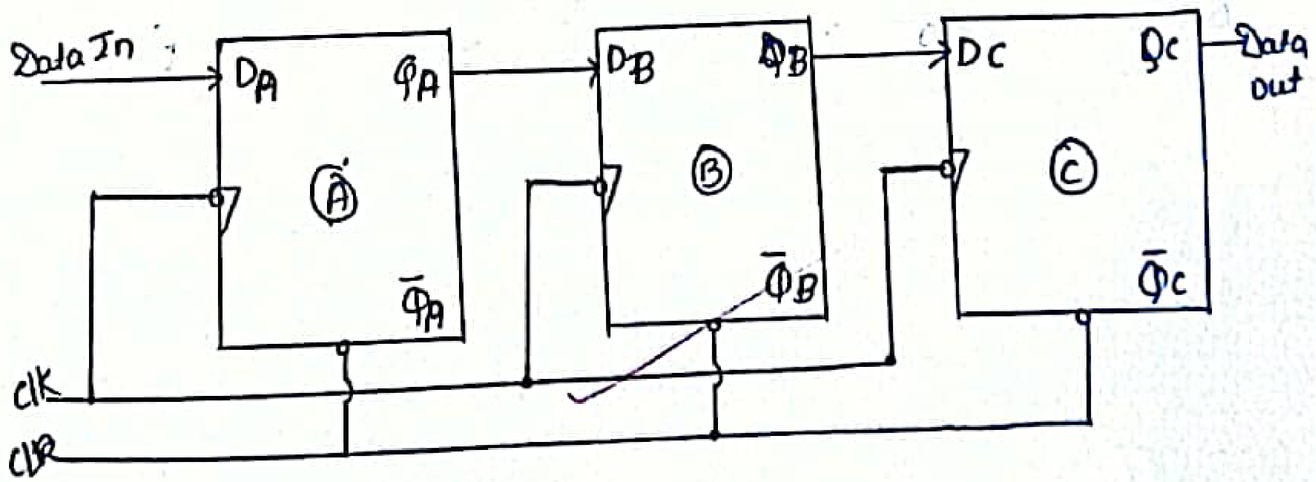
Objectives: To understand the concept of SISO register.

Lab Requirements: Digital Works Software

In SISO shift register data enters into a shift register serially, 1 bit at a time and data gets out of the register serially, 1 bit at a time.

Example: Design 3 bit shift register with its operational table and timing diagram.

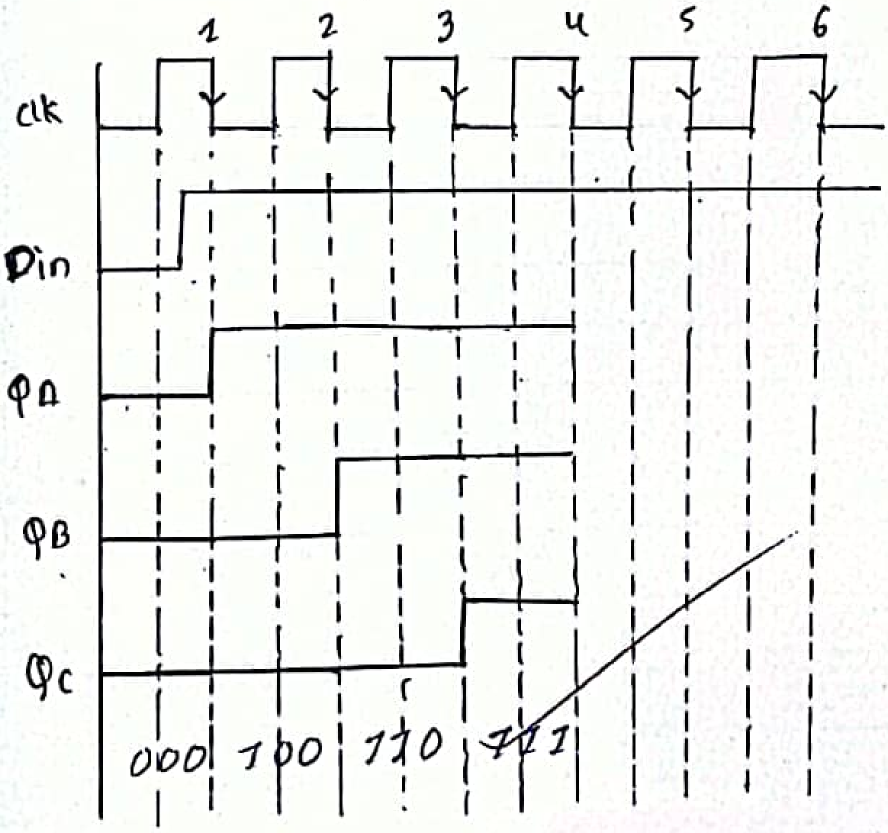
Logic Circuit Diagram:



Truth Table

Clk	ϕ_A	ϕ_B	ϕ_C
Initially	0	0	0
1st	1	0	0
2nd	1	1	0
3rd	1	1	1

Timing Diagram



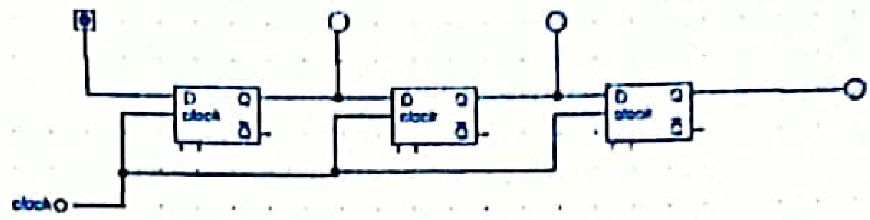
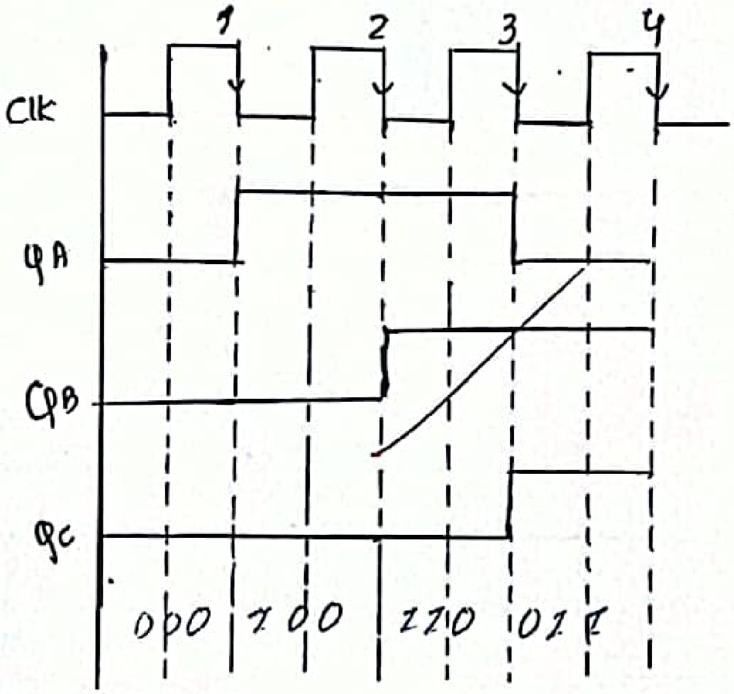


Fig: SISO Register

Truth Table

Clk	Q_A	Q_B	Q_C
Initially	0	0	0
1st	1	0	0
2nd	1	1	0
3rd	0	1	1

Timing Diagram



9·2 Serial In Parallel Out (SIPO)

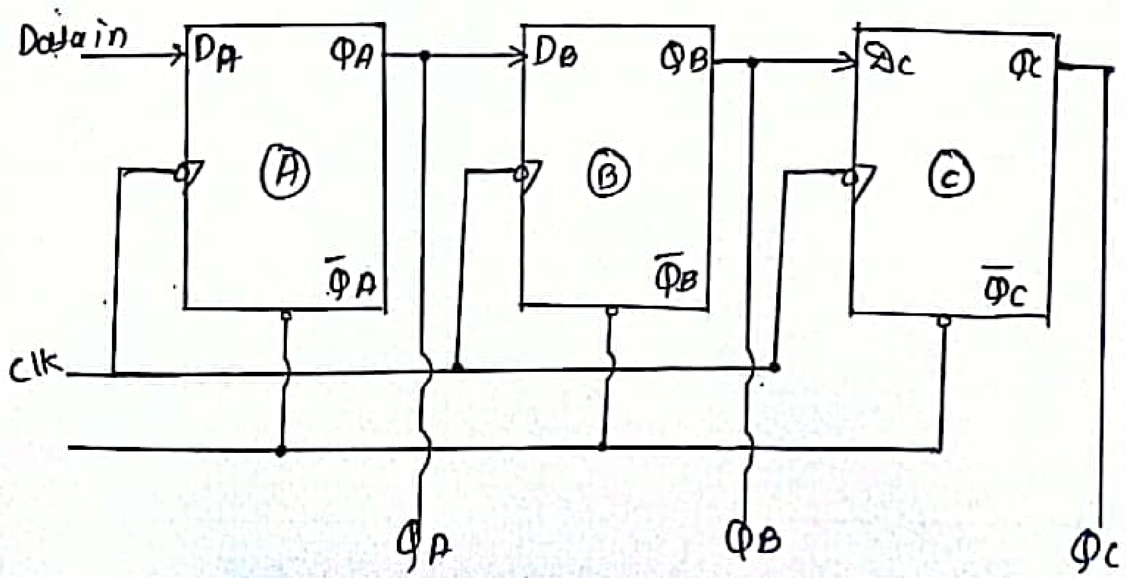
Objective: To understand the concept of SIPO shift register

Lab Requirement: Digital Works Software

In Serial In Parallel Out (SIPO) register, data enters into a shift register serially 1 bit a time and data out of a shift register parallely, all at a time. All of the flipflop are set and reset simultaneously.

Example: Design 3-bit SIPO Shift Register with its operational table & timing diagram

Logic Circuit Diagram:



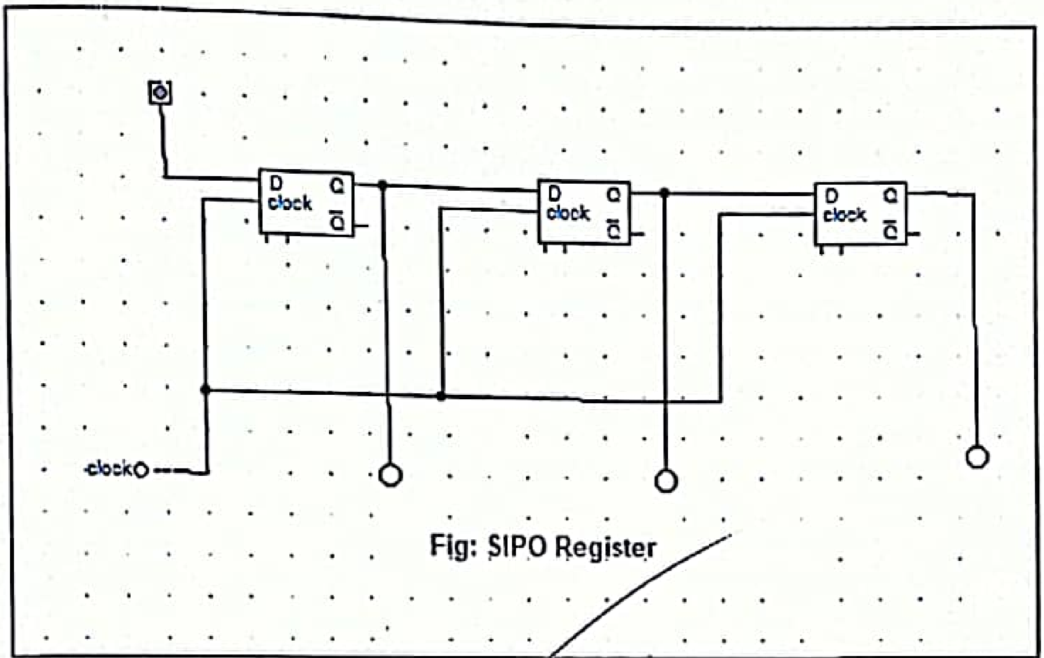


Fig: SIPO Register

9.3 Parallel In Parallel Out (PIPO)

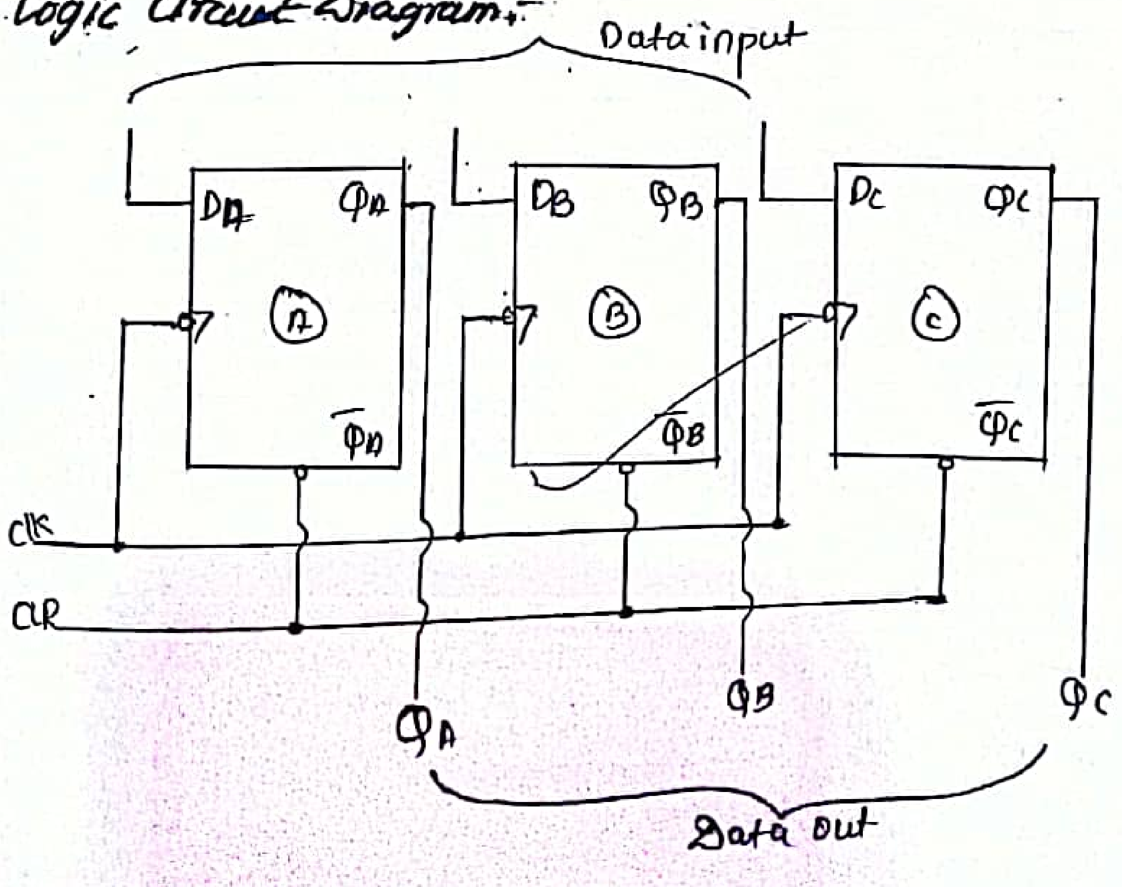
Objective: To understand the concept of PIPO Shift Register.

Lab Requirement: Digital Works Application

PIPO Register, here data enters into a shift register parallely all at one clock pulse and data out of the shift register parallely, all at a single clock pulse. All of the flipflops are set & reset simultaneously.

Example: Design 3 bit PIPO register with its operational table & timing diagram.

Logic Circuit Diagram:-



Truth Table

Clock	Q_A	Q_B	Q_C	
Initial	0	0	0	→ Data (Initially clear)
1st	0	1	1	→ Data in Parallely
	0	1	1	→ Data out Parallely

Timing Diagram:

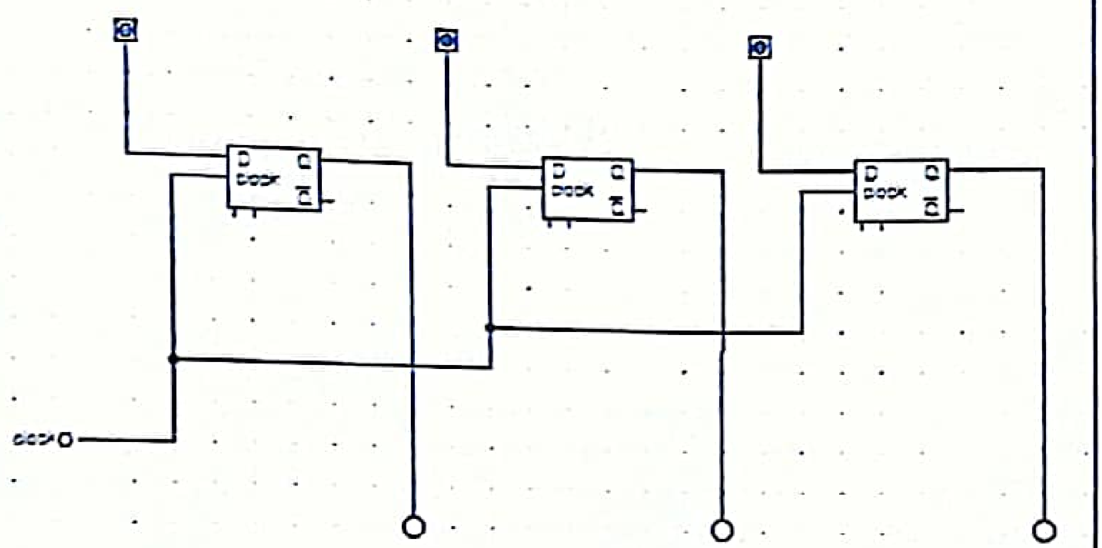


Fig: PIPO Shift Register

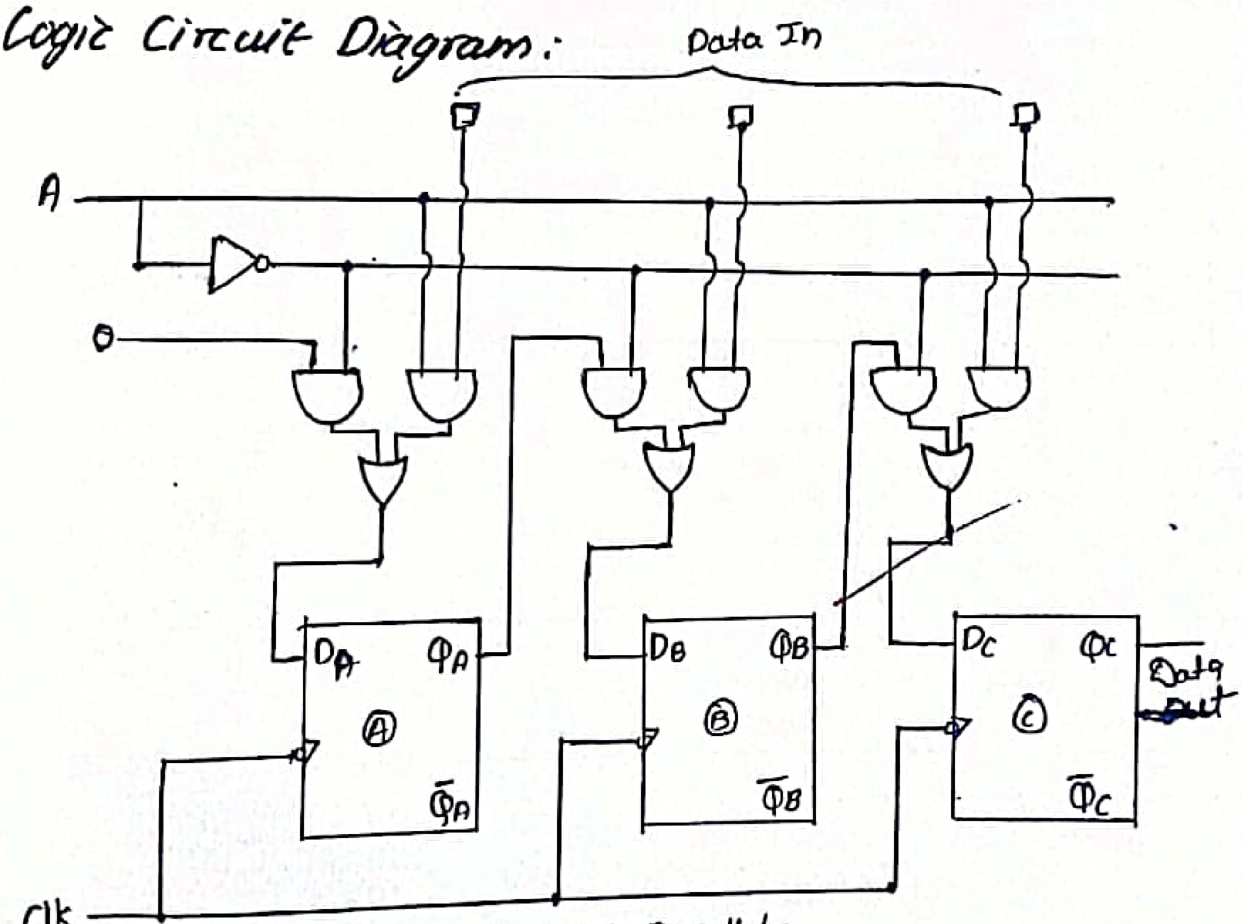
9.4 Parallel In Serial Out (PISO)

Objective: To understand the concept of PISO shift register.

Lab Requirements: Digital Works Software

In PISO shift Register, data enters into a ~~cont~~ shift register serially, all at one clock pulse and data out of the shift register serially one bit at a time. All of the flipflops are triggered by a common clock pulse and set a reset simultaneously.

Logic Circuit Diagram:



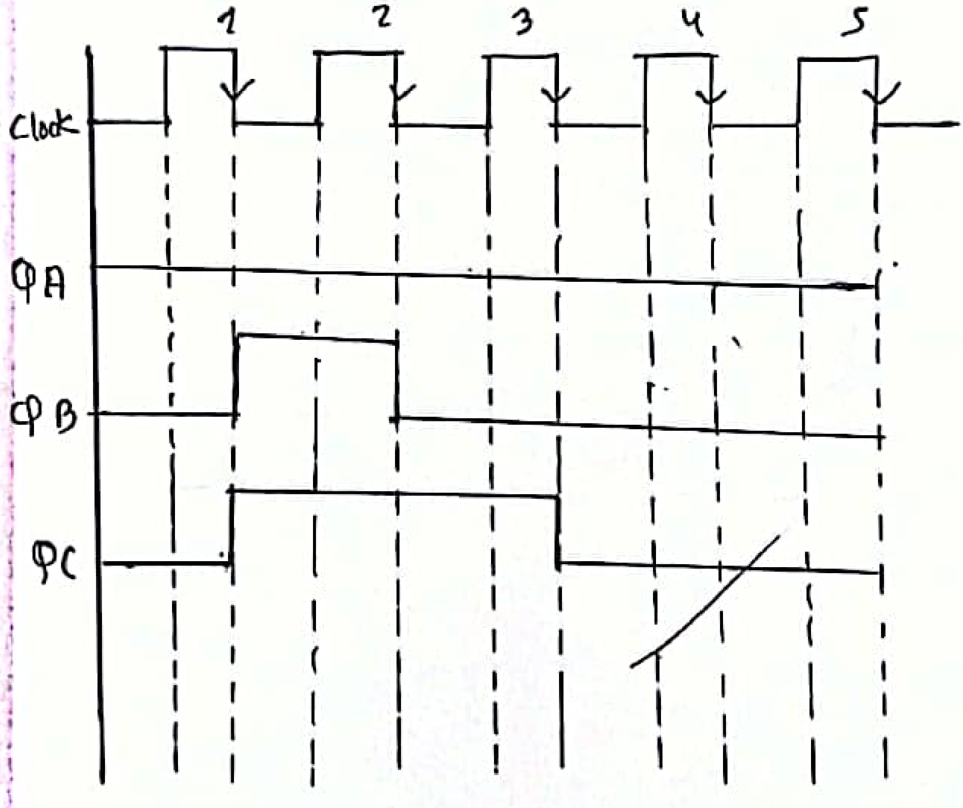
A = 1, Data In Parallely

A = 0, Data Out Parallely

Truth Table

Clock	A	Q_A	Q_B	Q_C		
Initially	X	0	0	0		→ Data Initially clear
1st	1	0	1	1		→ Data initially parallel
2nd	0	0	0	1	1	
3rd	0	0	0	0	1	→ Data Out Serially
4th	0	0	0	0	0	

Timing Diagram:



Experiment Diagram

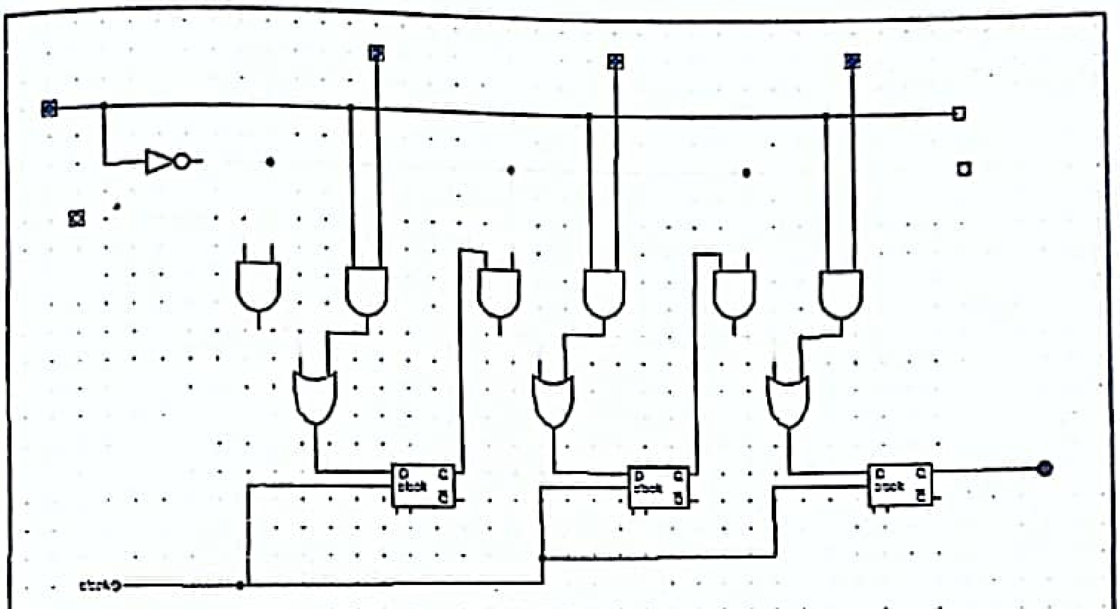


Fig: PISO Shift Register